"The Natural Language Inference Abilities of a Large Language Model."

Senior Honors Capstone Project

Submitted in partial fulfillment of requirements for the

Norbert O. Schedler Honors College

by

Kyle Andrew Urban

University of Central Arkansas

Conway, Arkansas

Spring 2026

Capstone Project Committee:

Mentor: Dr. Zachary K. Stine

Assistant Professor of Computer Science

Reader: Dr. Matteo Fiorini

Visiting Assistant Professor of Linguistics

Dean: Dr. Patricia Smith

Dean of the Schedler Honors College

Abstract

As Large Language Models (LLMs) see more use in daily life, it is important to know where their current shortcomings are when it comes to understanding human users. In this study, I analyze a large language model's performance on a task related to implication. I investigated two things. One, how a pre-trained but not fine-tuned LLM can accurately predict one of three types of logical implication? Two, what part do prepositions play in that prediction? We conducted our experiment using a dataset of 10k related sentence pairs – one instance with prepositions present and another absent. We created a classification task to sort sentences into three categories (entailment, contradiction, or neutral) based on underlying mathematical representation created by the BERT LLM. We found that the resulting accuracy scores never eclipsed 49% on either dataset — hardly better than random guessing. Scores did not improve even after modifying our experimental pipeline. These findings told us an untuned BERT cannot adequately make context predictions. To dive deeper into our research questions, I did a second experiment on a fine-tuned model which showed that textual relations from BERT contain latent signals that allow for a strong performance on classification tasks. Furthermore, the lack of spatial prepositions does not significantly affect task performance.

Linguistic Origins

The inspiration for this project originates from two key concepts in cognitive linguistics and semantics. The first idea is that of logical relationships.

For example, take the two sentences *p*, "The anarchist assassinated the emperor" and *q,* "The emperor died." These two phrases share a truth in a relationship known as entailment (Saeed, 2016). Essentially, this means that if *p* were true, then *q* would also have to be true since,

if the emperor was assassinated, he must be dead. Inversely, if *q* were false, then *p* would be false; ie: “the emperor isn’t dead so he wasn’t assassinated. However, *q* being true does not imply that *p* is true; ie: “the emperor died, but assassination was not the cause of death”.

Entailment is one type of logical relationship that plays a crucial role in language understanding. Another type is contradiction. Take sentences *p*, “Matteo stole my car” and *q,* “Matteo did not steal my car” (Saeed, 2016). Here, *p* is false when *q* is true and vice versa.

Finally, a third type is neutrality, which is actually not a type, but rather the lack of an implication. It can be thought of as when the logical relation between two sentences does not exist. For example, “I ate a sandwich” and “lemons are yellow,” have no bearing on each other’s truthfulness as statements. Implication, and its three main categories of entailment, contradiction, and neutrality, provide framing for the importance of our second key concept, landmark and trajectory (L&T).

In essence, L&T relates to spatial relations implied by a sentence, where the landmark is the reference point for a spatial movement and the trajectory is the movement itself. For example, in the sentence “St Paul’s Cathedral is over the bridge”, *over* is our trajectory (ie, spatial movement/relation) and *bridge* is the landmark with which that relation applies. L&T is important to the concept of implication because a trajectory-bearing word holds significant influence over what a sentence means. For example, the sentences “St Paul’s Cathedral is over the bridge” (fig. 1) and “the bird flew over the wall” (fig. 2) both contain the same trajectory word [over], yet the two sentences evoke different meanings of how that trajectory relates to our landmark (figs. 1 & 2).

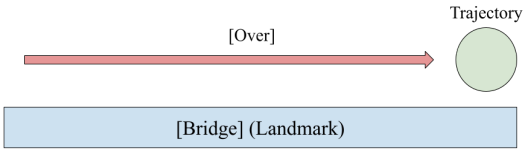


Figure 1: L&T Over the Bridge

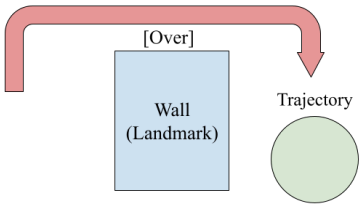


Figure 2: L&T Over the Wall

In the bridge example, [over] implies making physical contact with the landmark, whereas in the wall example, it implies a traversal around in an upward direction — but not making contact with — the landmark. In both sentences, the trajectory word summarizes the meaning of the other words in the sentence and provides the most core detail by what is intended in these kinds of spatial relation phrases.

In English, most trajectory words that encode spatial relations take the form of prepositions. Prepositions are a class of function words that combine with nouns to establish relations of location, direction, time, or manner of action.

One way that linguists test the kinds of information prepositions hold is to give people pairs of sentences containing prepositions and ask them to classify the type of implication (i.e., entailing, contradicting, or neutral) that pair displays. For example, given our previous two sample sentences, we may say that the phrase "My friends and I crossed the bridge to see St Paul's Cathedral" entails that "St Paul's Cathedral is over the bridge." Alternatively, "There was a dove that crashed into the stone barrier and died" contradicts the information encoded by "The bird flew over the wall."

These tests reveal more about the semantic impact of prepositions when they are removed. Let's try to classify our pairs again now with the prepositions removed. "My friends and I crossed the bridge to see St Paul's Cathedral" does not necessarily entail the statement **St Paul's Cathedral is the bridge*. In this new case, it rather gives the impression of neutrality.

Furthermore, the sentence q_{mod}, **the bird flew the wall,* doesn't give off an entailment relationship with *p* "there was a dove that crashed into the stone barrier and died" as compared to q_{unmod}, "The bird flew *into* the wall." In essence, a removal of the preposition can drastically confound the classification logical relationships between sentence pairs.

Computational Linguistics Origins

Switching gears, the year is 2026. Language is no longer restricted to people. We have spent the better part of this decade playing with a neat tool called Large Language Models (LLMs). While LLMs and people both share the ability of parsing language, LLMs do not share in the experience of understanding language. Instead, LLMs mirror what is colloquially called understanding by mathematically representing semantic information along with syntactic information. Thus, it is key to avoid the term 'understanding' when discussing LLMs. Rather, 'encoding' is a better fitting description of how a model engages with language. Specifically, LLMs will encode that language in a high dimensional Euclidean space known as latent space. The process of how this works requires a deeper dive into the functionality of LLMs

Machine Learning: Concepts, Terms & Practicalities.

To put the functionalities of LLMs into perspective, we must peel back a layer of pedagogy and address the field of Machine Learning (ML). ML is defined as the science of building various mathematical models to accomplish a task. In ML, all supervised learning tasks can be boiled down to either regression or classification. A model doing regression aims to predict a new numerical value whereas a model doing classification aims to predict what category a new piece of numerical data should fall into. For our experiments, we care about

doing a classification task. In these tasks, models make their choices categorize data based on its position in latent space. In the case of LLMs, that data happens to be language. Note, however, that 'input data' could be literally anything you want it to be: images, sound, bugs, etc., it doesn't matter. As long as it can somehow be represented mathematically and the type of model you choose is complex enough to handle it.

On the subject of model types, the LLM that I will be testing the performance of is the Bidirectional Encoder Representations from Transformers (BERT) model (Devlin, et al. 2018). The reasons for using the BERT model over a more name-brand model like the General PreTrained Transformer (GPT) model or Gemini are merely a decision of accessibility and simplicity. BERT is older and outdated compared to GPT, Gemini, etc, yet it is beginner friendly, free, and was quickest to implement in our programming environment – Google Colab.

Circling back to the idea that a model makes 'choices' based on data, there is a caveat here that will tie this all together. Of the many model types out there, BERT is an artificial neural network (ANN) (fig. 3). Recall the idea of the latent space. In simple terms, how an ANN works is that there are multiple layers of neurons. Each layer calculates a non-linear transformation of its inputs to generate predictions. The weights that define the transformations are estimated via gradient descent – which uses partial derivatives to adjust the weights to minimize predictive error.

Importantly, the final layer of neurons is known as the last hidden layer. The latent space it produces is known as the last hidden state. It is very important to how BERT encodes language. This is because BERT also has a transformer architecture (Delvin, et al. 2018). Essentially, this means that BERT learns to represent words according to dynamic positional

relationships (i.e; BERT notes the semantic reaction between words in its latent space by the distance between vectors)

As a rudimentary analogy, the last hidden state is akin to slicing open someone's brain to see their inner thoughts. The vectors and their locations in the latent space of the last hidden state are essentially the final product of BERT's encoding of input language data.

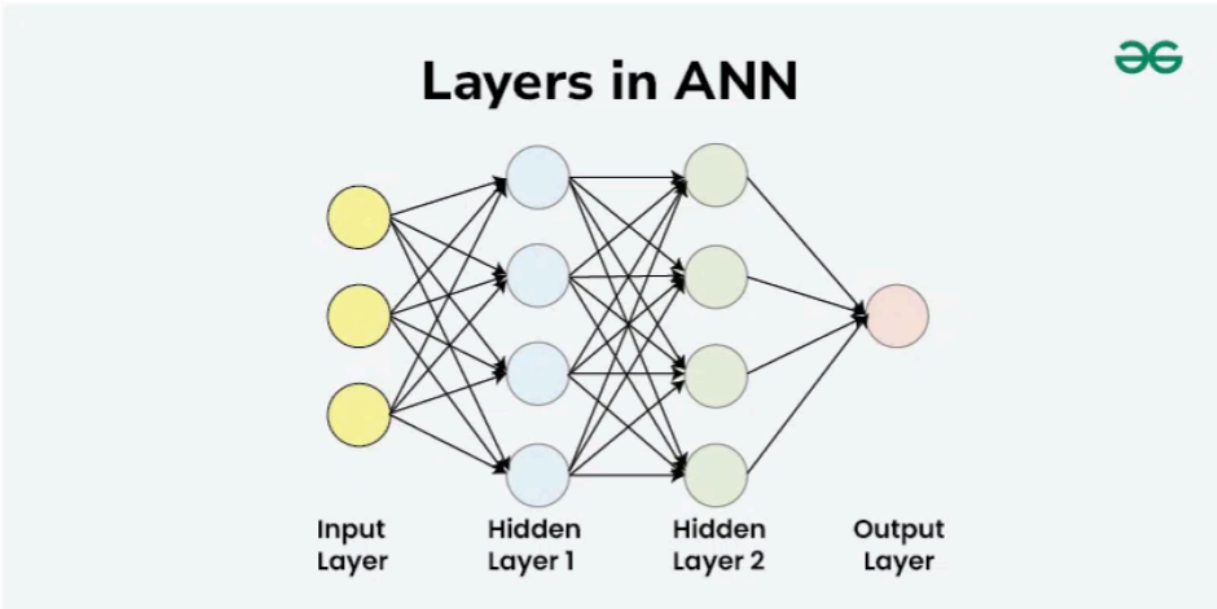


Figure 3.
An Artificial Neural Network (ANN)
(*GeeksforGeeks,* 2025)

With all this in mind, there is a layout for how ML concepts incorporate into the overarching linguistic questions being formed. A classification task exploiting latent space representation of the last hidden state that the BERT model generates on a control (prepositions present) and variable (prepositions absent) set of logically related sentence pairs can give an informative glimpse into how impactful prepositions are on BERT's encoding abilities. Two experiments spawned from this. The first one – involving an independent classification model —

and the second one — using BERT for both encoding and classification. Crucially, there exists a linguistic corpus providing the necessary data to test our hypothesis. Before we can run our experiments, we must take a dive to understand the data we are working with.

MultiNLI & Central Question

The MultiNLI is a corpus of 433k pairs of sentences that either entail, contradict, or neutralize (Bowman, et al. 2018). The idea is that this dataset can be used to score how well a model does in sorting sentence pairs into their correct implication type (a classification task). In turn, that score is a reflection of how well a model understands implication. The original MultiNLI contains a subset of sentence pairs that contain spatial prepositions. While the creators of the dataset were not necessarily concerned with the semantics of L&T, many instances of it are bound to be present in the dataset given its vast size. Furthermore, it is fairly easy to modify the sentence pairs themselves to remove words. Thus, the actionable steps of my research questions are born. Given a selected subset of the MultiNLI that only contains sentence pairs with an assorted list of spatial prepositions, how well does a model perform on classifying those pairs into implication types? How does the absence of those prepositions from the pairs affect the model's performance compared to when they are present? What does a difference, if any, in performance between the presence and absence scenarios say about the model's understanding of implication; i.e., how informationally relevant are the prepositions?

To design the experiments to test our theory, there is some additional information about our selected corpus that can aid us. For reference, the MultiNLI contains the correct label for the type of implication in each sentence pair. In the documentation left by the authors, this is known

as the gold_label. This acts as our answer key to accurately grade the model's performance when we test it.

Furthermore, the types of implication in the MultiNLI are one-way relationships; i.e., in a sentence pair, sentence one can entail, contradict, or be neutral to sentence two, yet not the other way around. Due to this, it is important to note that the first sentence will be more informationally dense than the second sentence and usually have a higher word count. In every pair, the first sentence is referred to as the premise, and is usually referred to as p or S_1. The second sentence is referred to as the hypothesis, and is commonly labelled as h or S_2.

Finally, the MultiNLI pulls language data from ten distinct genres of written or spoken English samples (Bowman, et al. 2018). The original makers of the corpus did this to provide a richer variance in the types of stylistic language that a model will have to work with. This is beneficial as it is more representative of how language is actually used. Some examples of the MultiNLI's sentence pairs are given in figure 4. It is important to note that due to limitations of downloading and using this full data corpus, I used a 10K subset of the MultiNLI that only contained five of the genres: fictional literature, government literature, slate poetry, telephone conversations, and travel brochures.

Premise	Genre Type **gold_label** annotator_labels	Hypothesis
Met my first girlfriend that way.	FACE-TO-FACE **contradiction** C C N C	I didn't meet my first girlfriend until later.
8 million in relief in the form of emergency housing.	GOVERNMENT **neutral** N N N N	The 8 million dollars for emergency housing was still not enough to solve the problem.
Now, as children tend their gardens, they have a new appreciation of their relationship to the land, their cultural heritage, and their community.	LETTERS **neutral** N N N N	All of the children love working in their gardens.
At 8:34, the Boston Center controller received a third transmission from American 11	9/11 **entailment** E E E E	The Boston Center controller got a third transmission from American 11.
I am a lacto-vegetarian.	SLATE **neutral** N N E N	I enjoy eating cheese too much to abstain from dairy.
someone else noticed it and i said well i guess that's true and it was somewhat melodious in other words it wasn't just you know it was really funny	TELEPHONE **contradiction** C C C C	No one noticed and it wasn't funny at all.

E = entailment, C = contradiction, N = neutral. (Bowman, et al, 2018)

Figure 4; MultiNLI sample

It is worth noting that the original authors of the corpus created the hypothesis sentences and gold_labels by hiring annotators (Bowman, et al., 2018). Sometimes, the annotators didn't agree on a gold_label for a pairing. In this case, the gold_label is the majority decision between 4 annotators. In rare cases, there were ties that got the gold_label of 'bar'; ie: "-". I culled these from the data set before running any experiments.

Experiment 1: Methodology

The methodology for the first experiment consisted of two parts. One, the obtention of BERT encodings of the last hidden state for each sentence pair for control and variable datasets. Two, the usage of those encodings as inputs for a separate classification model that sorts each encoding as representing a sentence belonging to either the entailment, contradiction, or neutral categories. Each of these parts were handled by their own separate Python file.

Getting the last hidden states first required a parsing of the MultiNLI data. To do this, I created a loop that would go through the data line by line, placing it into an array of class objects that would hold all the necessary data from the corpus; ie: row number, premise, hypothesis, and the gold_label for each sentence pair (fig. 5). The next step required in this process is known as tokenization. This is essentially where every word in the sentences is converted to a unique ID number. It is an important intermediary step that converts text data into numeric data that BERT can use (fig 6). For example, the word "in" may be converted to token ID "1999" or the word

```
#name of the multinli jsonl file
file_name = "multinli_1.0_dev_matched.jsonl"

path = "/content/drive/MyDrive/Colab Notebooks/DataforGetCLSofJust2sets"

#11/2/2025 additions (just to test if it can do "in" and then "inside") it

entries = []
#12/14/2025 --- Trying to mount google drive to see if it'll work this way.
with open(path + '/' + file_name, "r", encoding="utf-8") as file:
  for index, line in enumerate(file):
    data_line = json.loads(line)
    entry = MultiNLI_entry(
        row_num = index,
        sentence1 = data_line.get("sentence1", ""),
        sentence2 = data_line.get("sentence2", ""),
        gold_label = data_line.get("gold_label", ""),
    )

    entries.append(entry)
```

Figure 5: Pulling Data to Class Object

“outside” might be converted to “2648.” As such, a sentence such as "this example sentence in my paper is out of the box" would be tokenized as [2023, 2742, 6251, 1999, 2026, 3259, 2003, 2041, 1997, 1996, 3482]. In addition, BERT contains a special token ID of “101” reserved for the CLS vector. The CLS vector sits at the start of every sentence that BERT will read. It acts as a “summary” of BERT’s understanding of an entire sentence – this will be important later on.

```
Word: on top -- Token ID: [2006, 2327]
Word: in -- Token ID: [1999]
Word: out -- Token ID: [2041]
Word: over -- Token ID: [2058]
Word: under -- Token ID: [2104]
Word: above -- Token ID: [2682]
Word: below -- Token ID: [2917]
Word: inside -- Token ID: [2503]
Word: outside -- Token ID: [2648]
Word: around -- Token ID: [2105]
Word: behind -- Token ID: [2369]
Word: on -- Token ID: [2006]
Word: through -- Token ID: [2083]
```

Figure 6: Tokenized Words

The next step was to isolate pairs of sentences from the MultiNLI data that contained a preposition in the premise sentence. This consisted of searching the tokenized premises for any token IDs that matched those of our designated prepositions (fig. 6). Any pair of sentences that had a premise with one or more of the prepositions were saved, otherwise it was ignored. This was done to make sure our experiment was testing BERT only on relevant samples of spatial relation logic. In the end, this shaved our dataset from 10K to ~4.2K sentence pairs (fig 7).

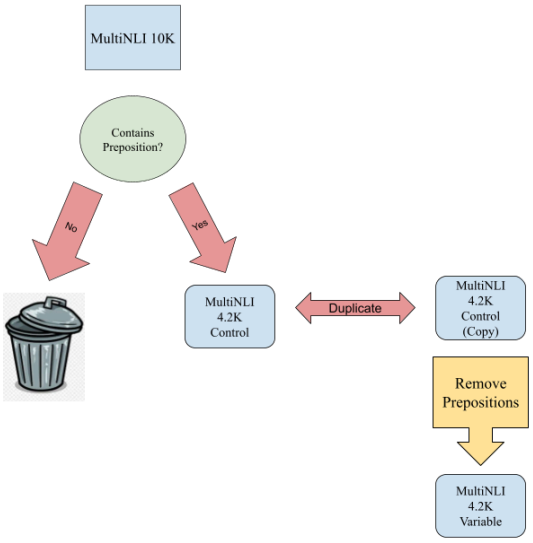


Figure 7: Control & Variable Creation

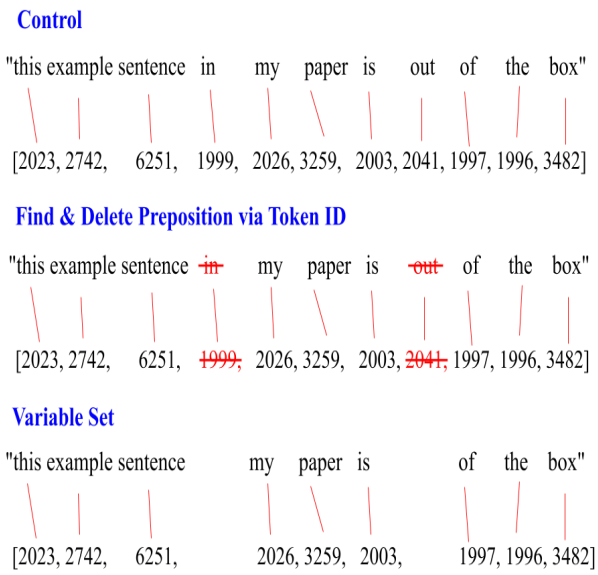


Figure 8: Removal via ID

After this, the 4.2K dataset was duplicated. One version would remain our control and the other would become our variable dataset (fig 7). To remove the prepositions from all premise sentences in the variable set, the same search strategy was used. Any sentence that had a token ID matching one or more of the prepositions would have that token ID removed. This is the semantic equivalent of taking the preposition out of a sentence entirely. In reference to our example sentence from earlier, taking “1999” and “2041” from the string [2023, 2742, 6251, 1999, 2026, 3259, 2003, 2041, 1997, 1996, 3482] carries the same meaning for BERT as removing the words “in” and “out” from the sentence “this example sentence in my paper is out of the box” does for us English speakers (fig 8).

The next step is the conversion of tokens to tensors — where we shed the TokenIDs as a shorthand label for our data and start using a mathematical representation of language that BERT can actually work with (fig 9). Essentially, each token ID is converted to a 768th dimensional

vector; ie: a list of 768 individual numbers that will represent a word in the latent space.

```
def convert_back_to_tensors(self):
  print("takes flattened tensors and makes them unflat")

  self.tensorized_s1 = tf.reshape(tf.convert_to_tensor(self.flatten_tok_s1, dtype=tf.int32), (1, len(self.flatten_tok_s1)))
  self.tensorized_s2 = tf.reshape(tf.convert_to_tensor(self.flatten_tok_s2, dtype=tf.int32), (1, len(self.flatten_tok_s2)))
```

Figure 9: Token to Tensor

This is only half the process though. Tensors are really only the containers for the mathematical representation (fig. 10). The conversion of tensors to embeddings is what gives us a suitable representation of language for BERT (fig. 10). Additionally, the embeddings act as the input layer for our classification model.

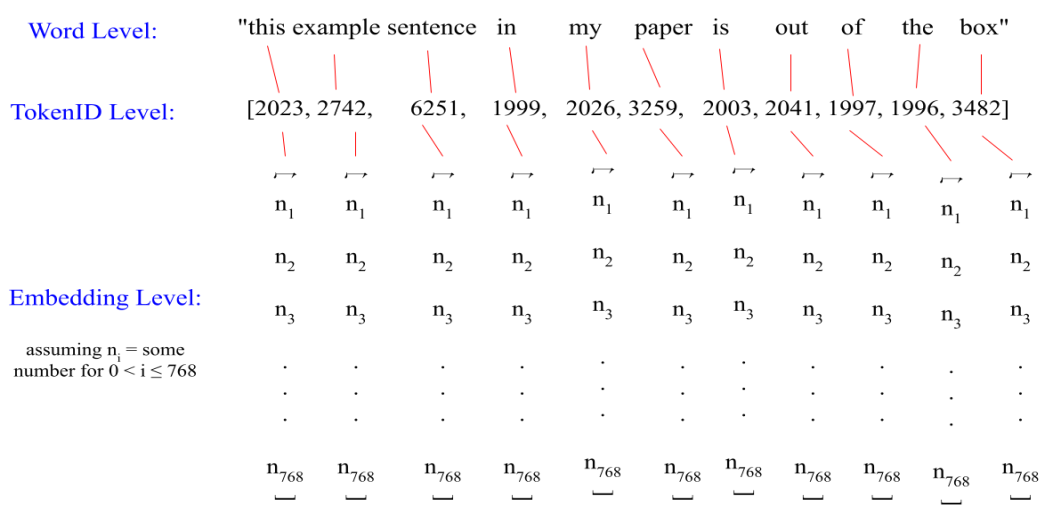


Figure 10: Embeddings

```
def embed_the_padded_tensors(self):
  self.embed_s1 = bert_model(self.pad_tensorized_s1, attention_mask = self.mask_s1) #1/10/2026
  self.embed_s2 = bert_model(self.pad_tensorized_s2, attention_mask = self.mask_s2)
```

Figure 11: BERT Model Embeds the Tensors

After the embeddings have been input to BERT, we can obtain the last hidden state. The last hidden state vectors are the end result of our embeddings going through several mathematical transformations across 12 layers of neurons (fig 12). This end result maps our

sentence pairs into the 768th dimensional latent space in a way our BERT model can adequately relate words together in a sentence as a measure of geometric distance between each vector (or point) in that latent space.

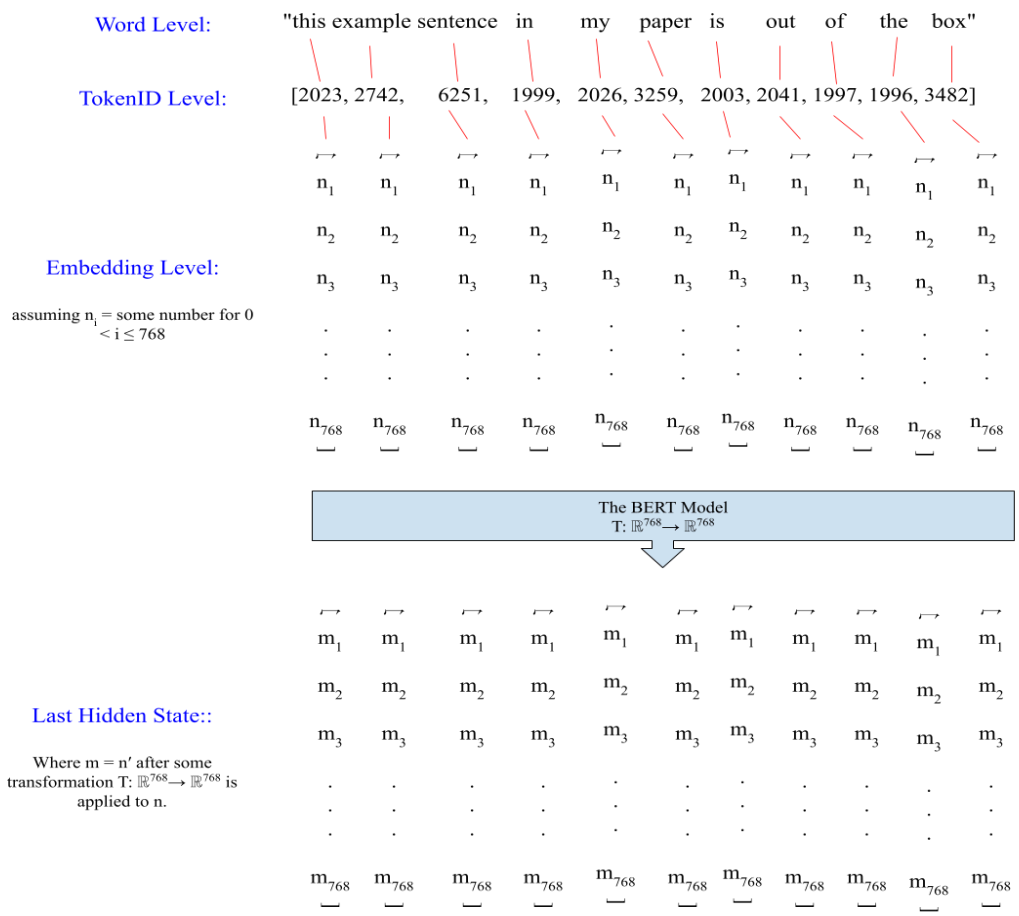


Figure 12: Last Hidden State

While the hidden state representation does encode BERT's language understanding, we must take an additional step to get representations of those encodings of our MultiNLI sentence pairs that is sufficient enough to use as inputs for a classification model. Recall from earlier that BERT contains an additional summation vector at the start of each sentence (fig. 13). Rather than using every last hidden state vector for every word of every sentence, we will instead rely on the

CLS summary vector to encode the meaning of each sentence; ie: two CLS vectors per sentence pair, one for the premise and another for the hypothesis (fig. 14).

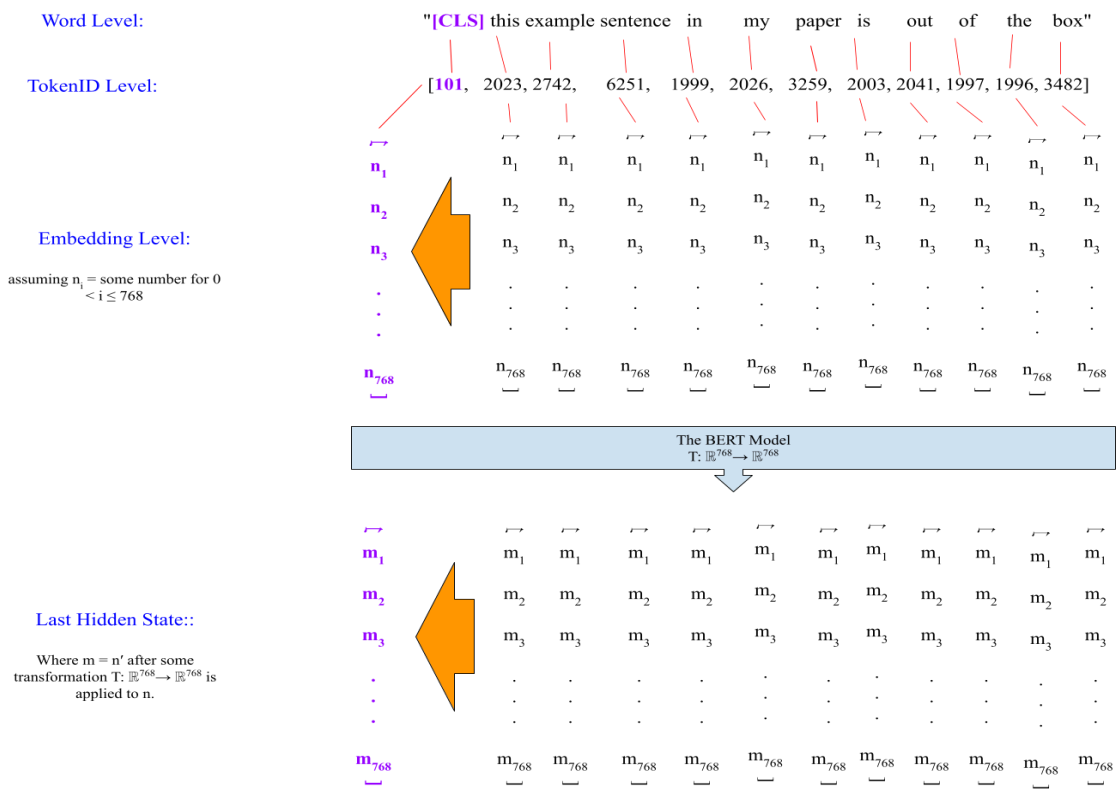


Figure 13: The CLS Vector

```
def get_cls_embeds(data_set):

  for entry in data_set:
    get_last_hid_state(entry)
    entry.cls_embed_from_last_hid_state()
    #print(entry.get_cls_embeds_rownum())
```

Loop repeats the process for every sentence pair in the dataset

Calls the class method

This class method pulls the CLS vector from the last hidden state of the premise and hypothesis sentences.

```
def cls_embed_from_last_hid_state(self):

  self.cls_embed_1 = self.last_hid_state_s1[:,0,:]
  self.cls_embed_2 = self.last_hid_state_s2[:,0,:]
```

Figure 14: Pulling CLS Vector from Last Hidden State

Once the CLS embeddings had been pulled for every sentence pairing of the control and variable datasets, they were used for training and evaluating our classification model. To train and evaluate a classification model for a task, we need input data. For the CLS vector embeddings to be used as input, they first had to undergo some additional changes to be a scientifically valid experiment. Following the model of other NLI research papers (Conneau, et

al. 2018), the CLS vectors of the premise & hypothesis were put together in a way that encoded the shared meaning between sentences of a pair. Specifically, the CLS vectors of the premise and hypothesis were concatenated with their element difference and element wise product to produce one singular combined vector (fig. 15). It is important to note that this was not the only strategy for combining BERT's embeddings into a meaningful input format for classification, additional methods used are noted later in the discussion section.

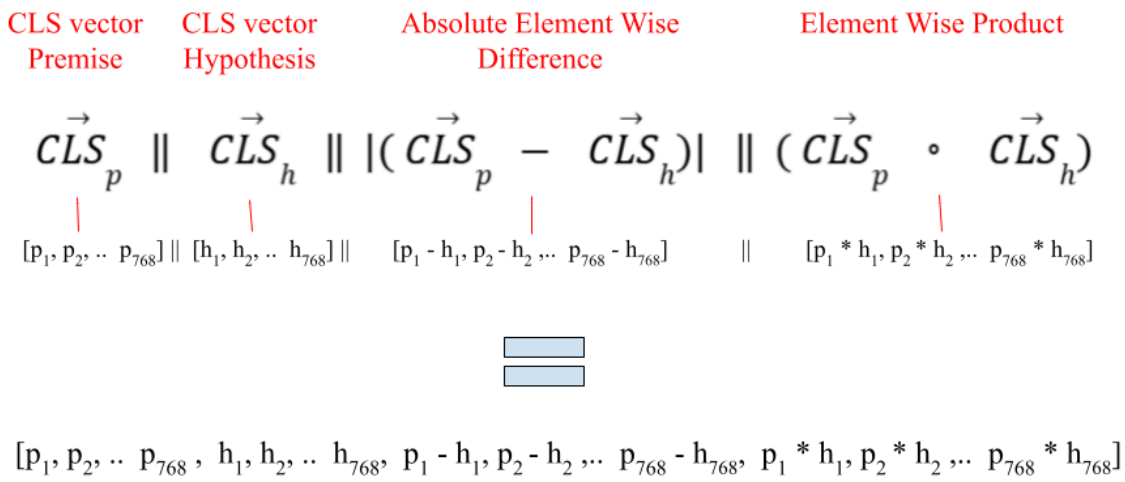


Figure 15: Column Wise concatenation of CLS vectors

Once this was done for both the datasets, we now have a single vector that represents each sentence pair. This will do nicely as input data for our classification model. Yet, to train a classification model, we also need to have labels that our model can learn to sort the input data into. Recalling our MultiNLI dataset, we are aware that each sentence pair came with a corresponding gold label; ie: entailment, contradiction, or neutral. Encoding these labels to a numeric value; ie: entailment = 0, neutral = 1, and contradiction = 2, makes them suitable for

training our classification model. Each sentence pair in our datasets are now represented by their combined CLS vector and its corresponding encoded gold label (fig. 16).

Every sentence pair in a dataset is now represented by the data structure....

[#, X, y]

Where..

- # is the index,
- X is the combined CLS vector,
- y is the numerically encoded gold label

Figure 16: Structure of Training Data

Our classification model will be trained and evaluated using stratified K-cross fold validation. Each dataset is cut into 10 segments, 9 of which will be used for training (fig. 17). The combined CLS vectors of each training subsets are put in a high dimensional euclidean space. The classification model then uses gradient descent to divide that space into three slices representing each of our gold label categories (fig. 18). During the training portion, the correct gold labels that came paired with each combined CLS vector are used as an answer key for the classification model (fig. 16). The model will continue to change its division of the space with each new round of gradient descent until a majority of the combined CLS vectors for that training subset are in a slice that corresponds to its correct gold label. In doing so, the classification model crafts a prediction function h(x) built for dividing that space (fig. 18). The nine training subsets make sure the prediction function can generalize to many different spatial distributions of combined CLS vectors (fig. 17).

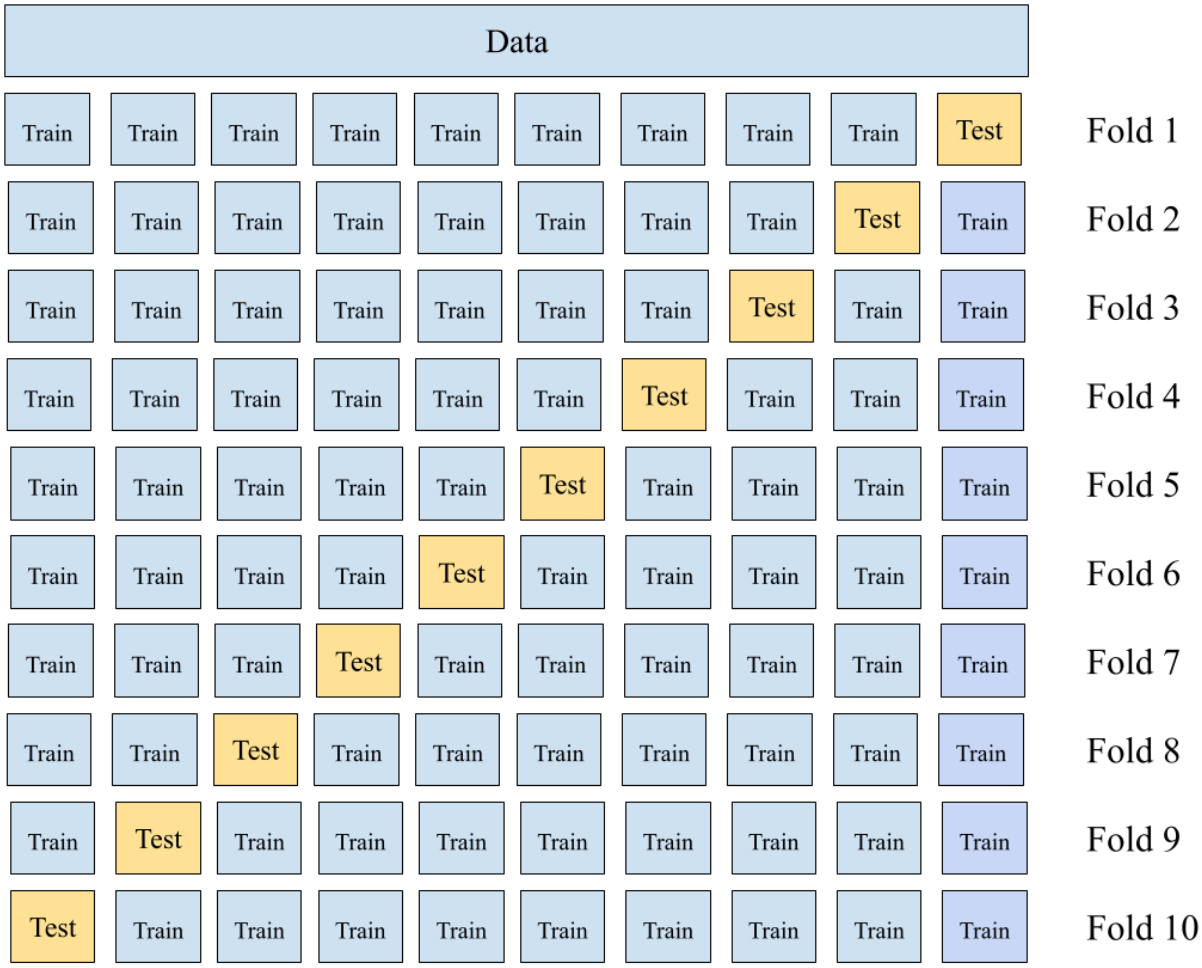
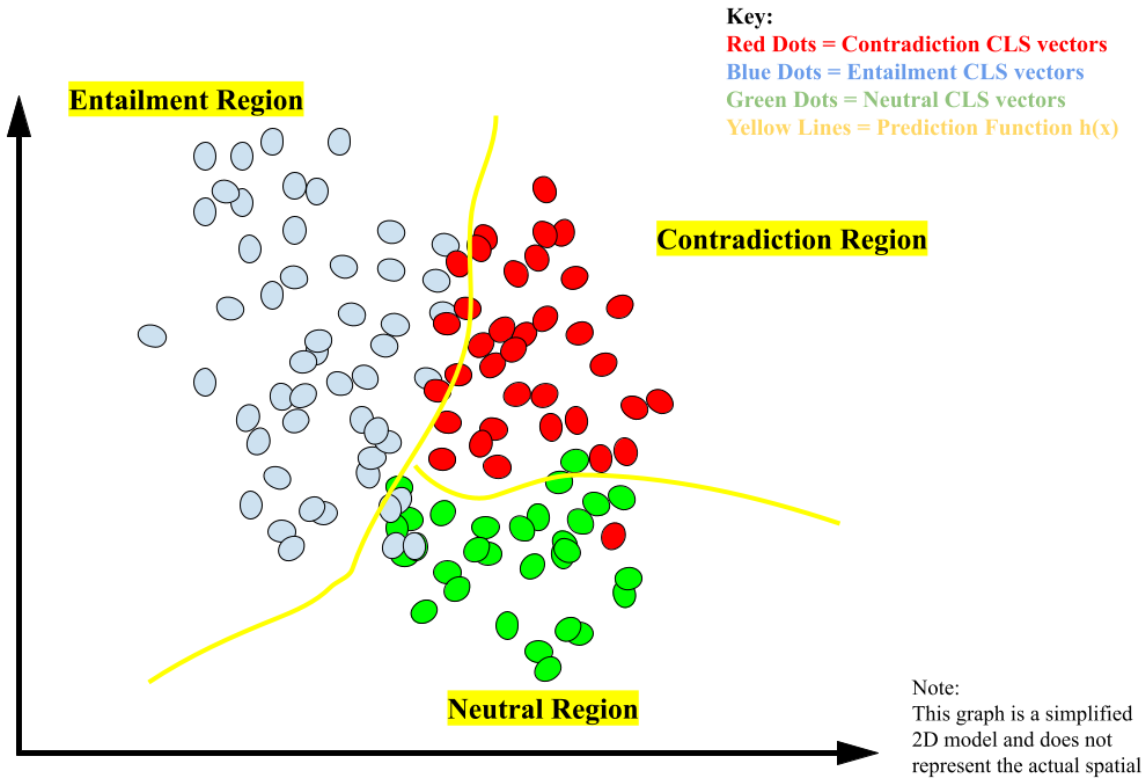


Figure 17: Stratified K Cross Fold



Note:
This graph is a simplified 2D model and does not represent the actual spatial division of our data by the model.

Figure 18: A Prediction Function dividing space to sort data into categories

Finally, the remaining evaluation subset is used to evaluate the prediction function h(x) of our classification model. The space containing the combined CLS vectors from the evaluation subset is divided by the prediction function h(x). The vectors are given prediction labels based on what slice they are in. Those prediction labels are then compared to the correct gold labels of those CLS vectors (fig. 16). The accuracy of the classification model for a given fold of stratified K-cross fold is based on the percentage of prediction labels that matched the correct gold labels for every vector in the space. For every fold, a different subset is held out as an evaluation set and a new prediction function h(x) is created via training with the remaining sets (fig. 17). This creates a more rigorous evaluation of the classification model as no two evaluation sets will be the same, so the model gets a fair test of its prediction abilities each time. Finally, an average of

all the accuracy scores from the evaluation of each fold is taken as a measure of the classification model's performance on a dataset. After doing a separate stratified K-cross fold on both the control and variable datasets, we arrive at two average accuracy scores that we can compare.

Experiment 1: Results and Discussion

The results of the first round of experiment one showed that the classification model was underfitting the data on all hyperparameter combinations (table 1). With accuracy scores hovering around 36~37%, the classification model did about as well as randomly guessing. This means that the classification model did not accurately predict the gold labels based on the combined CLS vectors it was given. However, this did not necessarily mean that BERT didn't encode the sentence pair data well enough to give the classification model sufficient data to predict labels. Yet, it does mean the modifications to the methodology were needed.

Concatenated CLS Avg Accuracy Scores					
Dimensionality Reduction?	Regularization	Solver	Solver Iteration	Control	Variable
No	L1	Liblinear	*	~36.9%	~37.3%
		Saga	*	~38.9%	~38.9%
	L2	Liblinear	*	~37.0%	~36.4%
		Saga	*	~36.6%	~37.3%
Yes (0.3)	L1	Liblinear	*	~36.0%	~35.5%
Yes (0.6)	L1	Liblinear	*	~39.0%	~38.8%
Yes (0.9)	L1	Liblinear	*	~46.3%	~45.5%
Yes (0.99)	L1	Liblinear	*	~42.2%	~42.8%
Yes (0.9)	L1	Saga	1000	~46.3%	~45.4%
		Saga	2000	~46.4%	~45.4%
* max solver iteration was not recorded for some trials.					

Table 1

Additional changes such as changes in regularization and solver strategy were made to the pipeline of the first experiment to improve the model's accuracy. Notably, PCA dimensionality reduction – which aligns the data along its eigenvectors to reduce noise – brought accuracy up to 45~46% for 0.9 dimensionality reduction (ie: the number of dimensions of the data was reduced to 90% of 768) with L1 regularization and SAGA solver (table 1). Unfortunately, that range of scores was too low to say anything conclusive.

While the results of round 1 weren't able to say anything about BERT's encoding abilities, they did tell us dimensionality reduction was key to increasing model accuracy and the importance of its inclusion in the pipeline. Furthermore, this also tells us that combined CLS vectors were not the best way to represent BERT's encoding.

The next round utilized a different combination of mathematical representations known as mean pooling. Instead of relying on the CLS vectors (fig 13) to provide a summarized vector of the entire sentence, mean pooling creates a more holistic representation of the sentence. In mean pooling, the last hidden state of each token is averaged together into a single vector (fig. 19), creating room for additional semantic influences that may have been drowned out by CLS vector representations. Though, it is important to note that the CLS vector is not abandoned with this strategy. The CLS vector itself is included in the mean pooling where it takes on a role as part rather than representation of the whole (fig 19).

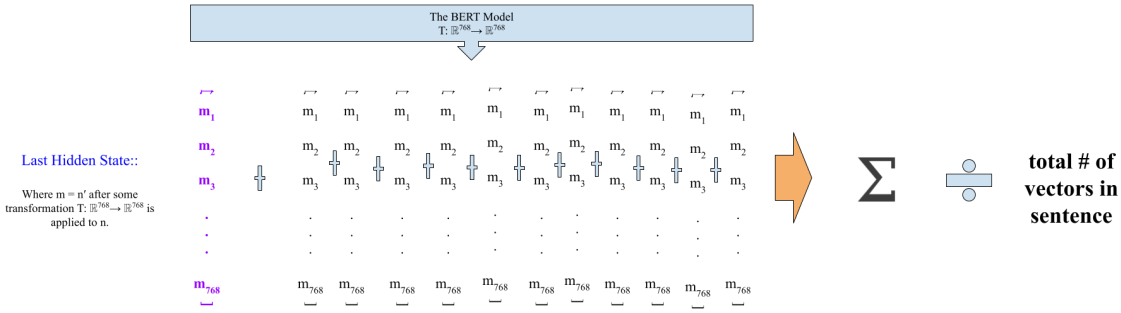


Figure 19: Mean Pooling

Reconcatenated versions of the datasets yielded slightly better performance across the board in the range of 40s and high 30s (table 2). The most optimal hyperparameter combination of 0.9 Dimensionality Reduction with the SAGA solver had a performance of ~48% for both datasets (table 2). This increase showed promise as a score of nearly half correct accuracy shows there is a semantic signal that the classification model can pick up on from BERT's encodings. Though once again, the model is underfitting the data.

Concatenated Mean Pool Avg Accuracy Scores					
Dimensionality Reduction?	Regularization	Solver	Solver Iteration	Control	Variable
No	L1	Liblinear	*	~ 40.1%	~ 40.3%
		Saga	*	~ 41.7%	~ 41.2%
	L2	Liblinear	*	~ 39.5%	~ 39.2%
		Saga	*	~ 40.2%	~ 39.3%
Yes (0.3)	L1	Liblinear	*	~ 36.2%	~36.2%
Yes (0.6)	L1	Liblinear	*	~ 37.1%	~37.4%
Yes (0.9)	L1	Liblinear	*	~ 48.0%	~48.2%
Yes (0.99)	L1	Liblinear	*	~43.4 %	~43.3%
Yes (0.9)	L1	Saga	1000	~ 48.0%	~48.3%
		Saga	2000	~ 48.0%	~48.3%
* max solver iteration was not recorded for some trials.					

Table 2

To make sure the cause of our inconclusive data was not that of a faulty hyperparameter, an additional round of experiments were done altering the inverse of the strength of regularization, represented by variable C (table 3 & 4). This additional hyperparameter probing was done on both the concatenated CLS vector (table 3) and concatenated mean pool vector (table 4) versions of the control and variable sets.

Concatenated CLS additional hyperparameter search						
Inverse of Regularization Strength (Hyperparameter C)	Dimensionality Reduction?	Regularization	Solver	Solver Iteration	Control	Variable
0.001	Yes (0.9)	L1	LibLinear	5000*	~35.3%	~35.1%
0.01					~45.3%	~44.6%
0.1					~46.3%	~45.8%
1					~46.3%	~45.5%
10					~46.3%	~45.4%
100					~46.3%	~45.4%
1000					~46.3%	~45.4%
0.001			Saga	200**	~35.7%	~35.6%
0.01					~45.5%	~44.9%
0.1					~46.1%	~45.5%
1					~46.3%	~45.4%
10					~46.4%	~45.3%
100					~46.4%	~45.4%
1000					~46.4%	~45.4%
1***		L2	LibLinear	5000*	~46.3%	~45.4%
			Saga	200*	~46.4%	~45.5%
*A max iter of less than 3000 for LibLinear would fail to converge						
**A max iter greater than 200 for Saga would be a waste of computational resources w/o noticeable effect						
***C=1 was the best hyperparameter, so other C values were not tested on L2 regularization to save computational resources						

Table 3

Concatenated Mean Pool additional hyperparameter search						
Inverse of Regularization Strength (Hyperparameter C)	Dimensionality Reduction?	Regularization	Solver	Solver Iteration	Control	Variable
0.001	Yes (0.9)	L1	LibLinear	5000*	~33.6%	~33.5%
0.01					~48.5%	~47.7%
0.1					~48.5%	~48.5%
1					~47.9%	~48.2%
10					~48.0%	~48.1%
100					~48.0%	~48.1%
1000					~48.0%	~48.1%
0.001			Saga	200**	~33.6%	~36.2%
0.01					~48.3%	~47.4%
0.1					~48.5%	~48.4%
1					~48.0%	~48.3%
10					~48.0%	~48.3%
100					~47.9%	~48.3%
1000					~47.9%	~48.3%
1***		L2	LibLinear	5000*	~47.9%	~48.3%
			Saga	200*	~48.0%	~48.0%
*A max iter of less than 3000 for LibLinear would fail to converge						
**A max iter greater than 200 for Saga would be a waste of computational resources w/o noticeable effect						
***C=1 was the best hyperparameter, so other C values were not tested on L2 regularization to save computational resources						

Table 4

Round 3 (table 3) showed that C=1, where regularization was reduced to nothing (table 3), had the best performance of all the combinations of hyperparameters. Though, ~46% & ~45% still show our model is underfitting. Similarly, round 4 (table 4) showed a performance of 48% for control and variable on the same combination of hyperparameters. Again, though, neither of these datasets provide any accuracy high enough to give conclusive results. Yet, these rounds did show the futility of playing around with many hyperparameter settings. This encouraged a narrowing of our pipeline for subsequent rounds to focus only on what made the largest difference in model performance, the data's representation.

```
def last_hid_state_from_embed(self):

  self.last_hid_state_s1 = self.embed_s1.last_hidden_state
  self.last_hid_state_s2 = self.embed_s2.last_hidden_state  #there

def last_hid_state_from_embed_joint(self):
  self.last_hid_state_joint = self.embed_joint.last_hidden_state

def cls_embed_from_last_hid_state(self):

  self.cls_embed_1 = self.last_hid_state_s1[:,0,:]
  self.cls_embed_2 = self.last_hid_state_s2[:,0,:]

def cls_embed_from_last_hid_state_joint(self):
  self.cls_embed_joint = self.last_hid_state_joint[:,0,:]
```

Figure 20: Joint Hidden State

Representing BERT's data with both concatenated CLS vectors and mean pooled vectors proved fruitful given the right hyperparameters and dimensionality reduction yet both had hit a ceiling. A revisit to how the BERT model functions, specifically its focus on next sentence prediction (Delvin, et al. 2018), showed that the concatenation methodology was flawed. BERT provides a better semantic encoding when the sentences it is working with are embedded

together (fig. 20). Thus, the artificial concatenation of separate vectors (fig. 15) (Conneau, et al. 2018) ceased in favor of directly using the joint embeddings of the premise and hypothesis sentences (fig. 21). Importantly, each token in the joint sentences were given token type IDs to differentiate the premise from the hypothesis (fig. 22). This made sure BERT encoded our joint sentences as two sentences existing in the same context rather than merely one long sentence (fig 22).

```
def get_cls_embeds_rownum_concate_gold_joint(self):
    print("d")
    joint = self.cls_embed_joint.numpy()
    joint = joint[0] #yes okay this is important
    return self.row_num, joint, self.gold_label
```

Figure 21: Abandoning Conneau's Method of Concatenation

```
remove_CLS_from_tensorized_s2()
#http://tensorflow.org/api_docs/python/tf/zeros_like token type ids are needed for the model to know that the two sen
s1_types = tf.zeros_like(self.tensorized_s1)
s2_types = tf.ones_like(self.tensorized_s2)
self.token_type_ids = tf.concat([s1_types, s2_types], axis=1)
#joint_tokens = (["[CLS]"] + sent_1_tokens + ["[SEP]"] + sent_2_tokens + ["[SEP]"])
#print("5")
#https://www.geeksforgeeks.org/deep-learning/tensor-concatenations-in-tensorflow-with-example/ gonna go with axis = 0
#self.tensorized_joint = tf.concat([self.tensorized_s1, self.tensorized_s2], axis = 0) #idk which axis I should do
self.tensorized_joint = tf.concat([self.tensorized_s1, self.tensorized_s2], axis = 1) #idk which axis I should do
#print("6")
```

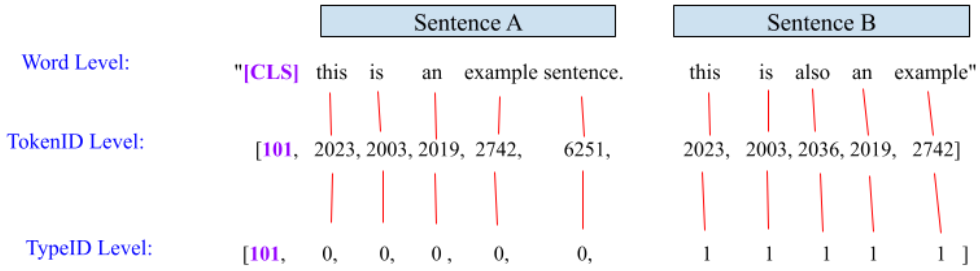


Figure 22: Type ID for Differentiating Sentences in Joint Embedding Process

The next round of experiments measured the performance of the classification model with both the jointly embedded CLS vector (table 5) and the mean pool of jointly embedded last hidden states (table 6). As illuminated by previous rounds of experiment one, a minimal amount of hyperparameter combinations were used. Therefore, both of these rounds only compared between solvers liblinear & SAGA and regularization strategies L1 & L2 (table 5 & 6).

Joint CLS Avg Accuracy Scores						
Inverse of Regularization Strength (Hyperparameter C)	Dimensionality Reduction?	Regularization	Solver	Solver Iteration	Control	Variable
1	Yes (0.9)	L1	LibLinear	5000	~50.6%	~51.1%
			Saga	200	~50.4%	~51.1%
		L2	LibLinear	5000	~50.5%	~51.1%
			Saga	200	~50.3%	~50.8%

Table 5

Joint Mean Pool Avg Accuracy Scores						
Inverse of Regularization Strength (Hyperparameter C)	Dimensionality Reduction?	Regularization	Solver	Solver Iteration	Control	Variable
1	Yes (0.9)	L1	LibLinear	5000	~44.3%	~44.8%
			Saga	200	~44.5%	~44.8%
		L2	LibLinear	5000	~44.5%	~44.8%
			Saga	200	~44.8%	~44.8%

Table 6

The joint mean pool displayed an average accuracy of ~44% for all outcomes (table 6), showing a downturn in the classification model's performance on joint embedding data. Clearly, the joint embedding being mean pooled actually hurt the representation of BERT's encodings. On the other hand, the 50~51% average accuracy with joint CLS vectors marked another significant upturn in the classification model's performance. This new rise in the classification model's accuracy assured that BERT is successfully providing enough mathematical information for a discernable pattern that can classify sentence pairs as entailment, contradicting, or neutral can emerge. However, this signal is not being picked up on by our classification model. Another round was conducted, this time Stratified K-Cross Fold was swapped for Random Forest Classifier (RFC). Given that the BERT model is a neural network, it is assumed its output would be non-parametric. Since Stratified K-Coss Fold uses logistic regression, a parametric model, it made sense the next course of action was to try out a non-parameteric model; ie: RFC, when we hit another performance ceiling with our original classification model.

Random Forest Classifier (Joint Mean Pooled) Avg Accuracy				
Pipeline Order	n components	n estimator	Control	Variable
Scale then PCA DimRed.	0.9	100	~35.0%	~34.1%
Scale no PCA DimRed.		500	~34.9%	~34.7%
PCA DimRed then Scale		100	~37.7%	~36.5%
		200	~37.7%	~38.4%
		500	~38.5%	~38.3%

Table 7

Unfortunately, RFC yielded accuracy scores worse than rounds 1 through 6 (table 7). Even with modifications to dimensionality reduction and scaling, RFC failed to provide the model complexity needed to fit the data.

This final round showed that our experiment was not going any further. Though we made great progress in showing that BERT encoded a pattern in the semantics of the language well enough to reasonably tease apart our three types of logical relations, the performance of our classification models was still too low. If there were any difference between the encodings of our control and variable sets, it would only be able to emerge if the accuracy was higher. Given that a non-parametric classification model, RFC, was not fit for seemingly non-parametric data, and the parametric classification model, logistic regression, had topped out, the next logical step was to redesign the experiment from the ground up. Instead of using an independent classification model, it would be best to investigate using BERT itself as the classification model.

Experiment 2: Methodology

The data preparation for experiment two was streamlined compared to the first experiment. The strategies of culling the data down to only sentence pairs that contained prepositions (fig. 7), as well as preposition removal (fig. 8), remained the same as the first experiment. Given the method of data representation that proved most effective during the first experiment, the BERT model then encoded the sentence pairs jointly (fig. 23) using the same token type ID strategy as the first experiment to help the BERT model differentiate the premise and hypothesis sentence (fig. 22).

```python
# This concatenates sentence 1 and 2 as a single sequence before getting the CLS embedding.
def tokenize(batch):
    return tokenizer(
        batch["sentence1"],
        batch["sentence2"],
        truncation=True,
        padding="max_length",
        max_length=128
    )
```

Figure 23: Joint CLS embeddings by batch

Unlike the first experiment where joint CLS embeddings were used as training and test data for a separate classification model, the second experiment involved training a BERT model on a subset of the control data (fig. 24)– a process known as fine-tuning. Then, the fine-tuned BERT model was tested on the classification task using the opposing control subset and its corresponding variable subset (fig. 24). The sentence pairs in the variable set, which were counterpart to the subset of sentence pairs in the control data, were excluded from the test set to keep the evaluation of model performance fair between the test sets (fig 24).

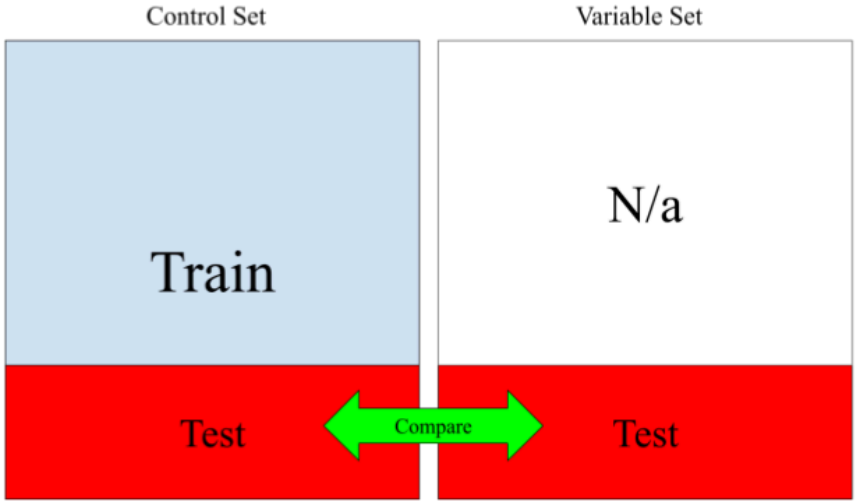


Figure 24: Preventing Data Leakage

Also unlike experiment 1, which only had 13 prepositions being removed from the premise sentence (fig. 6), experiment 2 had a list of 25 prepositions being removed (fig. 25). This was done to broaden the scope of our test on the types of L&T our classification task would end up covering.

```
def deswords_tok(tokenizer):
  words_to_remove = ["on top", "in front", "next to",
                     "in", "out", "over", "under", "above", "below", "inside",
                     "outside", "around", "behind", "on", "near", "through",
                     "throughout", "beneath", "into", "beside", "off", "between",
                     "next", "upon", "from"]

  words_remove_tok_ids = [tokenizer.encode(word, add_special_tokens = False) for word in words_to_remove]
  print("Token Ids of words to remove: ", words_remove_tok_ids)
  return words_remove_tok_ids
```

Figure 25: Extended Preposition List

Additionally, to make sure the experiment was valid over multiple trials, each train-test split of the data was randomized with a seed (fig. 26). The average accuracy score between ten evaluation trials over both the control and variable subsets, each with ten separate yet equal fine-tuned BERT models, provided two scores with which BERT's ability to encode logical relations could be compared.

```
#https://stackoverflow.com/questions/73154119/how-do-i-convert-pandas-dataframe-to-a-huggingface-dataset-object
dataset = Dataset.from_pandas(entries)


dataset = dataset.train_test_split(test_size=0.1, seed = seed)
train_dataset = dataset["train"]
#I need to steal the train_indices from the train_dataset
train_indices = train_dataset['original_index']
eval_dataset = dataset["test"]
print("train_test_split done")
```

Figure 26: Random Seed + Train/Test Split

Experiment 2: Results

The results of the second experiment were accurate enough to provide a conclusive answer to our research question. The average accuracy of fine-tuned BERT on the classification task on the control set was ~69.3% (table 8). In comparison, the average accuracy for the validation set was ~68.4% (table 9). Being nearly in the 70% accuracy range, these scores are high enough to be conclusive. The difference between our control and variable was only that of 1.1% percent, not enough to be significant. Reasonably, we can conclude that the lack of spatial prepositions caused minimal change in BERT's ability to encode logical relations between sentences.

Control Data						
Average Accuracy: 69.33%						
Trial	Loss	Acc	Runtime	Samp/s	Steps/s	Epoch
1	83.14%	67.44%	0.955	498.708	31.431	4
2	76.82%	69.12%	0.964	494.019	31.136	4
3	97.83%	66.81%	0.982	484.548	30.539	4
4	95.63%	66.81%	0.958	496.725	31.306	4
5	93.66%	68.91%	0.979	486.144	30.639	4
6	67.80%	76.05%	0.967	492.512	31.041	4
7	80.56%	70.17%	0.961	495.229	31.212	4
8	91.91%	68.70%	0.972	489.799	30.870	4
9	83.30%	69.96%	0.963	494.278	31.152	4
10	68.91%	69.33%	0.984	483.719	30.486	4

Table 8: Control Avg Accuracy & Intertrial Accuracy Scores for Experiment 2

Variable Data						
Average Accuracy: 68.42%						
Trial	Loss	Acc	Runtime	Samp/s	Steps/s	Epoch
1	82.67%	67.44%	0.903	527.061	33.218	4
2	77.80%	67.02%	0.935	509.090	32.085	4
3	98.30%	66.81%	0.932	510.876	32.198	4
4	97.72%	65.76%	0.934	509.913	32.137	4
5	95.74%	68.70%	0.931	511.511	32.238	4
6	69.78%	74.37%	0.940	506.637	31.931	4
7	82.20%	69.54%	0.935	509.196	32.092	4
8	91.61%	66.60%	0.939	507.089	31.959	4
9	83.36%	70.17%	0.936	508.442	32.045	4
10	70.79%	67.86%	0.933	510.092	32.149	4

Table 9: Variable Avg Accuracy & Intertrial Accuracy Scores for Experiment 2

Additionally, for those that are curious, the training loss, hyperparameters, and randomization seeds are provided (table 10). These show that our fine-tuned model's loss function was able to converge such that our model fit the data. Furthermore, it shows that our test and training data did not have data leakage. These qualities demonstrate that the test scores for the control and variable sets are both conclusive and valid, meaning we can draw a conclusion from experiment two's results.

Training Loss from Control Data						
Avg Loss: 0.5779773558670683						
Trial	Loss	Steps	Runtime	Samp/s	Steps/s	Epoch
1	57.20%	1072	130.616	131.163	8.207	4
2	57.10%	1072	136.072	125.904	7.878	4
3	60.53%	1072	137.441	124.650	7.800	4

4	58.01%	1072	138.352	123.829	7.748	4
5	54.25%	1072	140.198	122.199	7.646	4
6	59.61%	1072	137.945	124.195	7.771	4
7	56.71%	1072	140.782	121.692	7.615	4
8	59.65%	1072	140.604	121.846	7.624	4
9	56.58%	1072	138.550	123.652	7.737	4
10	58.35%	1072	138.855	123.381	7.720	4
Configuration & Hyperparameters					**Trial Seed Map**	
Setting		Value			Trial	Seed
Epochs		4			1	43
Logging Steps		1			2	44
Frozen Layers		0			3	45
Train Batch Size		16			4	46
Eval Batch Size		16			5	47
FP16 Status		TRUE			6	48
weight_decay		0.01			7	49
Small sample set skipped		Yes			8	50
					9	51
					10	52

Table 10: Additional Information for Experiment 2

Linguistic Implications

The negligible difference that the absence of spatial prepositions had on BERT's ability to encode logical relations is open to interpretation. However, there are a few key takeaways for those in the field of linguistics. One, BERT is able to encode logical relations despite lacking L&T features. This is probably due to BERT's attention layers, which allow the model to embed a sentence with a glimpse of its surrounding context (Vaswani, et al. 2023). This is thanks to recursive layers in BERT's ANN, where the embeddings of surrounding tokens influence the embedding of the current token due to being included in the weight calculations that create those

embeddings. Therefore, the removal of a word from a sentence does not throw off BERT, as the model builds its representation of language in a way that is holistic and semantically preserving.

Two, this could also mean that prepositions may not be the most semantically relevant element when it comes to L&T. Building on the first key point, we can posit the removal of prepositions may not have been enough to disrupt BERT's attention layers. This doesn't mean that BERT's attention layers are unassailable. However, it does demonstrate that prepositions are not the part of a sentence that can trip up BERT on logical relations. Perhaps there are other elements of L&T sentences, such as the verbs, that may prove more effective in disrupting BERT's attention layers.

Finally, it is important to note that these results and their interpretations are not set in stone. The experiments came from the perspective of a first ever thesis project of an undergraduate student and machine his learning journey. It is full of trial and error which necessitates future polishing. Linguistically, BERT's NLI abilities are only one small piece of the puzzle for how LLMs operate and alone does not provide commentary on how language itself functions, yet rather only how our models perceive it.

Limitations

No experiment is without its limitations. This project had four main constraints that hindered its scope and impact. First and foremost was time. Work on experiment one began in December of 2024 and wasn't completed until January of 2026. In addition, I started this project knowing nothing about Python or Machine Learning. Several times did entire portions of the experiment have to be scrapped during the learning curve. Oftentimes, tasks ended up taking much longer than originally imagined. Problems were usually more complex than initially

envisioned and many times life got in the way. Despite this, a solid rigour of eight hours a week got the job done.

In addition, experiment two remains incomplete in two key ways. One, it did not have a regularization strategy. It is possible that performance could have been higher if given the time to explore various regularization strategies to help the fine-tuned BERT better handle the evaluation data it was given. Two, it is also possible that BERT was not the best model for the task. If the second experiment were to be fully comprehensive, we would have fine-tuned and evaluated our datasets on multiple LLMs using a set of regularization strategies. Although, every project must eventually come to a stopping point. This is not a weakness however, yet merely room for further study in the future. Again, given the time constraints for completing my honors project, this was the furthest I could expand on the second experiment.

Finally, our experiment was limited by the size of the dataset used. Our experiments only used the 10K subset of the MultiNLI corpus. While this made the project manageable and computational feasible for the computational resources we had access to, not using the full 433k corpus may have left the models unable to fit the data. Contemporary experiments that used the entire corpus were shown to have performance percentages in the mid 70s to high 80s (Bowman, et al. 2018). It is very well possible a larger difference in performance could have occurred between the control and variable sets if more data was used.

Addressing Limitations

Of course, all these limitations to the machine learning elements of my capstone might affect the linguistic implications of the results. If properly addressed, this could further enlighten upon BERT's NLI abilities beyond the scope of the experiment. Introducing an optimizer like

PyTorch's AdamW package would go to properly address the lack of regularization. Expanding the palate of models to include BERT-large, SBERT, RoBERTa, BERT_nli, MBERT or perhaps other models not based on BERT might yield better accuracy scores where any potential performance difference could have a chance to manifest. Getting away from using Google Colab would allow for the usage of the full corpus. Though this would require setting up a programming environment equipped to handle large machine learning projects. If these three limitations could be addressed in a follow-up extension of experiment two, this capstone project would yield a fuller insight on BERT's abilities.

Conclusion

In conclusion, we set out to discover if the absence of spatial prepositions affected the BERT LLM's ability to encode logical relations. Through the course of two experiments that broke open the mathematical engine behind the model, we were able to determine that the answer was no. Spatial prepositions do not have an effect on the model's ability to encode logical relations. For linguists, this means that BERT's attention mechanics skirt around the issue of missing words, though this is open to interpretation. For machine learnists, this conclusion is not complete. The limitations of this experiment leave many points for further follow up and experimentation. For me, this project helped me learn the basics of how LLMs function, the skills needed to work with them, and how to blend my two fields of study. I am very thankful to all those that helped me along the way and I recognize that I have much more to learn. Overall, our experiments provide a conclusive answer to a niche question in computational linguistics while leaving additional room for expansion in the future.

Works Cited

Bowman, R. Samuel, et al. “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference” *ArXiv.* 19 Feb, 2018. https://doi.org/10.48550/arXiv.1704.05426

Conneau, Alexis, et al. “Supervised Learning of Universal Sentence Representations from Natural Language Inference Data” *ArXiv.* 8 Jul, 2018. https://arxiv.org/pdf/1705.02364

Delvin, Jacob, et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” *ArXiv.* 11 Oct, 2018. https://doi.org/10.48550/arXiv.1810.04805

Evans, Vyvyan. & Green, Melanie. “Cognitive Linguistics: An Introduction.” *LAWRENCE ERLBAUM ASSOCIATES.* 2006. pg 334-339

“Layers in Artificial Neural Networks (ANN)” *GeeksforGeeks.* 17 Dec, 2025. https://www.geeksforgeeks.org/deep-learning/layers-in-artificial-neural-networks-ann/

Saeed, I. John. “Semantics” *WILEY Blackwell.* 4th ed. 2016. pp 44-45; 94-97.

Vaswani, Ashish, et al. “Attention Is All You Need” *ArXiv.* 2 Aug, 2023 https://arxiv.org/pdf/1706.03762